Algorithm Architecture Specification and Software Engineering Proposal

Revised Milestone 1 | November 4 2026

Metadata field	Project team specification
Project title	Oak Sports Academy Training Registration and Management System Enhancement
Team and section	Oak Sports - Group 5 | III-BCSAD
Lead Algorithm Architect	Nathaniel Babalcon
Backend and Systems Engineers	Hexel John Ventura; Rommel Villanueva
Data and QA Engineer	Jurie Aplaca
Systems Analyst and Technical Lead	Aaron Abiera
Repository URL	https://github.com/hex-xel/oak-sports-academy
Revision date	November 4, 2026; final submission date to be confirmed by the team

Course: ALGOCOM (Algorithms and Complexity) / SOFTENG (Software Engineering)

Summary

This revised proposal replaces preference-weighted schedule recommendations with a CART decision tree for student churn risk, supported by a participant lifecycle finite state machine and magic-byte image validation. These algorithms support the revised assessment, enrollment, attendance, renewal, and document workflows. The report specifies the algorithms themselves, their mathematical models and complexity, service boundaries, implementation backlog, and validation plan.

CART estimates future participation lapse. The FSM enforces confirmed lifecycle events. Model settings and quality thresholds are explicit engineering proposals; no measured accuracy or runtime is claimed.

Report scope: Tasks 1 through 6 and references. The presentation deck will be prepared separately by the team.

Revision basis and report guide

Source authority and scope

The client flow dated November 4, 2026 defines the revised business workflow [1]. The supplied Milestone 1 handout defines the academic report requirements [2]. The existing technical specification provides the current project baseline [3]. This report proposes a revision; it does not change specifications.md, application behavior, or the repository architecture.

The existing schedule recommendation algorithm needs preference scores and an optimization feature that are absent from the revised flow. Students still choose frequency and weekday or weekend schedules; those choices alone do not establish a recommendation feature. CART instead supports the separately required Trial Conversion and Student Churn Prediction capability. Decision trees belong to an explicitly permitted course paradigm [2, 5].

Algorithm	Concrete role	Academic position
CART binary classification tree	Learn splits from labeled participation histories; infer future lapse risk.	Primary advanced supervised learning algorithm.
Lifecycle FSM	Validate registration, trial assessment, approval, enrollment, inactivity and withdrawal events.	Professor-required supporting algorithm.
Magic-byte and MIME validation	Identify JPEG or PNG upload signatures and reject metadata mismatches.	Professor-required supporting algorithm; paired with bounded image screening.

Report guide

Section	Coverage
Task 1	Problem formulation, feature vector, label, constraints and certificates.
Task 2	CART learning and prediction, FSM transitions, upload validation, examples.
Task 3	Training and inference derivations, auxiliary memory, scale dimensions.
Task 4	Decoupled architecture and typed request and response contracts.
Task 5	User stories, engineering tickets, correctness and benchmark plans.
Task 6	All six academic defense questions and technical justifications.
References and appendix	2021–2026 literature and standards; specification reconciliation.

The current specification already includes the FSM and magic-byte algorithms in Section 15 [3]. Its direct-enrollment transition, 14-day post-trial inactivity rule, and

constant-time description of the broader image pipeline need reconciliation with the revised flow and the analysis below. These discrepancies are recorded in the appendix for owner approval.

Task 1 Computational problem modeling

Operational problem

Academy staff need an accurate record of each participant's assessment and enrollment progress, safe collection of valid-ID images, and an explainable way to prioritize follow-up when participation may lapse. The revised flow records attendance, package progress, assessment results and renewal reminders [1]. These supply relevant operational events. Educational dropout research supports investigating classification for follow-up, but university findings do not establish accuracy for a sports academy [6].

Primary classification model

For each weekly snapshot time t of an ENROLLED participant, construct x from events committed at or before t. Use one row per participant per selected week. Exclude approved medical pauses and records with incomplete follow-up from the training cohort. Exclude names, document images, guardian details, gender, addresses, assessment decisions and future events from predictors.

Feature	Definition at snapshot t	Valid range
x1	Days since most recent attended session	Integer ≥ 0
x2	Consecutive unexcused absences	Integer ≥ 0
x3	Approved package sessions remaining	Integer ≥ 0
x4	Attended / eligible sessions among the last up to 10 scheduled sessions; omit approved medical excusals	Real in [0, 1]; denominator > 0
x5	Days since current enrollment approval	Integer ≥ 0

Proposed label: y = 1 if there is neither an attended session nor an approved renewal during (t, t + 30 days]; otherwise y = 0. This is a future 30-day participation-lapse proxy, not proof of permanent withdrawal. Only snapshots whose entire outcome window is observable receive labels. The 30-day horizon and medical-pause exclusion require academy confirmation before production use.

$$D = \{(x_i, y_i)\}_{i=1}^{n},\ x_i \in \mathbb{R}^5,\ y_i \in \{0,1\}$$

$$fT(x) = 1\ if\ \hat{p}T(y = 1 \mid x) \geq \tau;\ otherwise\ 0$$

The computational task is binary classification. Training greedily reduces weighted Gini impurity under depth and minimum-leaf constraints; it does not solve the globally optimal decision-tree problem [5]. Proposed settings are d = 5, maximum

depth H = 8, minimum leaf size L = 5 and $\tau = 0.5$. Tune only on training and validation data; report the chosen settings with the model version.

<!-- AI Assistance: ChatGPT assisted with the feature formulation, proposed label and classification objective. Team review is required before submission. No empirical results are claimed. -->

Task 1 Constraints and output certificates

Constraints and data readiness

Registration must pass email verification, required guardian details for minors and waiver acceptance. Both newly white and colored belts take trial assessment before enrollment. Staff mark assessment approval or rejection, and an accepted student must enroll within one week or the account is deleted [1, pp. 1–3]. The formal deadline boundary in Task 2 is an implementation proposal; its timezone and exact cutoff must be confirmed.

White belts remain restricted to group classes and module prerequisites; colored belts receive the relevant class options without modules. The flow gives ten sessions per package, medical make-ups subject to approval, and forfeiture for unexcused absences [1, pp. 4–7]. The algorithms consume these authoritative decisions and never infer eligibility or alter session balances from a risk score.

Train only when both classes exist and participant-disjoint chronological training, validation and test partitions can be formed. Proposed minimum: 100 mature snapshots from at least 50 participants, with at least 20 positive and 20 negative examples overall, and both classes in each partition. This is a feasibility gate, not a guarantee of statistical adequacy. Increase data collection if held-out uncertainty remains high.

Output certificates

Engine	Success output	Controlled alternative
CART training	Versioned tree, feature order, thresholds, class counts, limits, cohort cutoff and validation metadata.	INSUFFICIENT_DATA; no deployable predictor.
CART prediction	Risk label, leaf class frequency, support count and traversed feature tests.	NOT_SCORED for incomplete features or model absence.
FSM	Previous and next state, accepted event, version and deadline when applicable.	INVALID_TRANSITION, GUARD_FAILED, STALE_EVENT or replayed result.
Image validation	Detected MIME, byte count, dimensions, checksum and opaque storage key after screening.	INVALID_SIGNATURE, MIME_MISMATCH, SIZE_LIMIT, MALFORMED_IMAGE or QUALITY_REVIEW.

Cold start behavior

Without an eligible validated model, return mode RULE_BASELINE and probability null. A proposed baseline flags FOLLOW_UP_REQUIRED when the consecutive unexcused absence count is at least three; otherwise return NO_RULE_TRIGGER. This baseline is transparent and requires academy confirmation. It is not presented as learned prediction, and absence of a flag does not establish low risk. Synthetic data may demonstrate the CART implementation but cannot establish academy predictive validity.

No classifier result changes login access, academy approval, payment status, enrollment, or the one-week deletion policy. Those remain deterministic backend responsibilities [1, 3].

Task 2 CART strategy and split mathematics

Schema field	Engineering specification
Algorithm name	Classification and Regression Trees style binary classification using Gini impurity.
Design paradigm	Supervised learning with greedy recursive binary partitioning.
Foundational reference	Raschka, Liu and Mirjalili (2022), classification trees [5]. Complexity notation follows Cormen et al. (2022) [4].
Baseline alternative	Majority-class prediction and the documented absence rule.
Computational justification	Prefix class counts evaluate all legal split boundaries in a sorted feature in linear scan time, avoiding a full recount at every boundary.
Invariant	Each node owns a disjoint row-index range; child ranges partition the parent; all leaves meet minimum size.

Split criterion

For a node holding m examples with c positive labels, define p = c/m. For feature j and threshold a, left contains xj ≤ a and right contains xj > a. Consider only boundaries between distinct observed values where both children have at least L rows.

$$G(m,c) = 1 - (c/m)^2 - (1 - c/m)^2$$

$$Gain(j,a) = G(m,c) - (mL/m)G(mL,cL) - (mR/m)G(mR,cR)$$

Select the largest positive gain. Use feature order x1 through x5 and ascending thresholds to break ties deterministically. A threshold is the lower observed value at a boundary; this avoids midpoint overflow. Stop at depth H, a pure node, fewer than 2L rows, or no gain greater than $\varepsilon = 10^{-12}$. Each leaf stores positive count, sample

count and empirical positive frequency; this frequency is not a calibrated probability guarantee [5].

Worked split example

For illustration only, use L = 1 and six synthetic examples with x2 = [0, 0, 1, 3, 4, 5] and labels [0, 0, 0, 1, 1, 1]. The parent has G = 0.5. Splitting at x2 ≤ 1 makes both children pure, so weighted child impurity is 0 and gain is 0.5. A new x2 = 4 follows the right branch, yielding empirical frequency 3/3 = 1. This demonstrates mechanics; six artificial rows cannot justify an operational accuracy claim.

<!-- AI Assistance: ChatGPT assisted with the CART specification, split equations and worked example. Team review is required before submission. No empirical results are claimed. -->

Task 2 CART training algorithm

The following specifies the actual training algorithm rather than only its paradigm. X and y are immutable arrays. A single index array I is partitioned in place; temporary feature pairs are discarded before recursion. Binary label counts make each candidate evaluation constant time [4, 5].

```
TRAIN_CART(X, y, H=8, L=5, epsilon=1e-12):
  validate nonempty rectangular X, finite values, binary y
  require H >= 0, L >= 1, n >= L; both classes for deployment
  I <- [0, 1, ..., n-1]
  return BUILD(0, n, 0)

BUILD(lo, hi, depth):                    // I[lo:hi] owns node
  m <- hi-lo; c <- sum(y[I[r]] for lo <= r < hi)
  leaf <- {positive:c, count:m, frequency:c/m}
  if depth = H or c = 0 or c = m or m < 2*L: return leaf
  parent <- G(m,c); bestGain <- 0; best <- NONE
  for j <- 0 to d-1:
    A <- [(X[I[r]][j], y[I[r]], I[r]) for lo <= r < hi]
    sort A ascending by (featureValue, rowIndex)
    leftPos <- 0
    for k <- 1 to m-1:
      leftPos <- leftPos + A[k-1].label
      if k < L or m-k < L: continue
      if A[k-1].value = A[k].value: continue
      rightPos <- c-leftPos
      gain <- parent - (k/m)*G(k,leftPos)
                     - ((m-k)/m)*G(m-k,rightPos)
      if gain > bestGain + epsilon:
        bestGain <- gain; best <- (j, A[k-1].value)
      // Equal gains keep first feature and first boundary.
    discard A
  if best = NONE: return leaf
  (j,a) <- best
  mid <- PARTITION_IN_PLACE(I,lo,hi, X[row][j] <= a)
  // Partition uses two pointers; swaps advance toward each other.
  left <- BUILD(lo,mid,depth+1)
  right <- BUILD(mid,hi,depth+1)
  return {feature:j, threshold:a, left:left, right:right,
          positive:c, count:m}
```

G is defined with the prediction pseudocode below. A gain must exceed the current best by ε; gains within ε are treated as tied and preserve the first feature and boundary. Each feature sort uses the row index as a secondary key for reproducibility. Training returns a deployable model only after the data-readiness and evaluation gates have passed.

<!-- AI Assistance: ChatGPT assisted with the training pseudocode, deterministic tie handling and stopping rules. Team review is required before submission. No empirical results are claimed. -->

Task 2 Prediction and correctness

```
PREDICT(tree, x, threshold=0.5):
  if tree is absent: return {status:NOT_SCORED, reason:NO_MODEL}
  if x has missing, nonfinite or out-of-range fields:
    return {status:NOT_SCORED, reason:INVALID_FEATURES}
  node <- tree; path <- []
  while node has children:
    j <- node.feature; a <- node.threshold
    wentLeft <- x[j] <= a
    append (j,a,wentLeft) to path
    node <- node.left if wentLeft else node.right
  frequency <- node.positive / node.count
  label <- HIGH_RISK if frequency >= threshold else LOWER_RISK
  return {status:SCORED, label, leafFrequency:frequency,
          support:node.count, path, modelVersion:tree.version}

G(m,c):
  if m <= 0: reject INVALID_NODE
  p <- c/m
  return 1 - p*p - (1-p)*(1-p)
```

Why the algorithm terminates

Every accepted split has two nonempty children, each smaller than its parent and each containing at least L rows. Recursion also increases depth toward H. Consequently training terminates. Prediction takes exactly one child at each internal node until it reaches a leaf, so it terminates within H tests for the proposed bounded tree.

What the algorithm guarantees

At a node, the scan considers every distinct-value threshold for each feature that satisfies the minimum-leaf constraint. Prefix counts equal the number of positive labels in the current left prefix, and subtraction gives the right count. Thus the scan selects the largest measured local gain subject to numerical tolerance and tie rules. The partition preserves exactly those memberships. This is a local greedy guarantee; it does not guarantee the globally best tree or future predictive accuracy [5].

The returned path and support count allow staff to inspect why a follow-up flag appeared. A small leaf can produce an extreme frequency, so the certificate exposes support rather than presenting a score as certainty. A record outside the validated cohort must return NOT_SCORED instead of forcing an unsupported estimate.

Separation from lifecycle decisions

A HIGH_RISK result is advisory. Only an authorized event with valid business guards can move the FSM. Trial approval remains a staff assessment decision. The current account authorization rules remain in force until the owner approves reconciliation with the revised flow's login wording [1, 3].

<!-- AI Assistance: ChatGPT assisted with prediction logic and the termination and local correctness arguments. Team review is required before submission. No empirical results are claimed. -->

Task 2 Lifecycle finite state machine

Define a deterministic partial transition function δ(s,e,g) → s', where s is lifecycle state, e is an authenticated event, and g is its business guard. Finite-state transition modeling follows [12]; unlisted state/event pairs are rejected. The table is the proposed revised transition specification based on the assessment flow [1]; it supersedes neither the live code nor specifications.md until approved.

Current state	Event and required guard	Next state
`REGISTERED`	TRIAL_SCHEDULED by staff; verified registration, guardian and waiver requirements complete.	`TRIAL_SCHEDULED`
`TRIAL_SCHEDULED`	ASSESSMENT_ATTENDED; coach/staff attendance recorded for the scheduled trial.	`ASSESSMENT_PENDING`
`ASSESSMENT_PENDING`	ASSESSMENT_APPROVED by authorized staff; set acceptedAt and deadline.	`APPROVED_AWAITING_ENROLLMENT`
`ASSESSMENT_PENDING`	ASSESSMENT_REJECTED by authorized staff.	`REJECTED`
`APPROVED_AWAITING_ENROLLMENT`	ENROLLMENT_CONFIRMED before deadline; eligible class/module/package, academy approval and required fee confirmation.	`ENROLLED`
`APPROVED_AWAITING_ENROLLMENT`	DEADLINE_REACHED; now ≥ deadline and no timely confirmed enrollment.	`DELETION_DUE`
`ENROLLED`	INACTIVITY_CONFIRMED by approved policy or staff; medical pauses excluded.	`INACTIVE`
`INACTIVE`	RENEWAL_CONFIRMED; eligible renewal approved under current academy rules.	`ENROLLED`
`ENROLLED`	RENEWAL_CONFIRMED; approved renewal and balance update handled separately.	`ENROLLED`
`Any nonterminal state`	WITHDRAWAL_CONFIRMED by authorized staff after participant/guardian request.	`WITHDRAWN`

REJECTED, WITHDRAWN and DELETION_DUE are terminal in this initial proposal. Reapplication, rejected-account retention, trial cancellation/rescheduling, and trial no-shows require academy rules before adding transitions. The revised flow does not confirm the former two-trial-day entitlement, so this report does not impose it.

DELETION_DUE is an internal cleanup instruction, not successful deletion. The account-removal service must execute the client's deletion policy and report completion. Exact deletion scope and any permissible anonymized reporting or audit retention require owner approval. Lifecycle state, account access status and payment status remain separate fields.

<!-- AI Assistance: ChatGPT assisted with the proposed FSM state set and transition table. Team review is required before submission. No empirical results are claimed. -->

Task 2 Lifecycle processing and conversion

```
APPLY_EVENT(snapshot, event, now):
  require authenticated actor authorized for event type and participant
  begin transaction; lock participant; load current state and version
  if event.id already committed:
    load stored certificate; end transaction; return certificate
  if event.expectedVersion != current.version: reject STALE_EVENT
  rule <- fixedTransitionTable[current.state, event.type]
  if rule absent: reject INVALID_TRANSITION
  if not rule.guard(current,event,now): reject GUARD_FAILED
  next <- rule.target
  if next = APPROVED_AWAITING_ENROLLMENT:
    acceptedAt <- authoritative assessment approval timestamp
    deadline <- acceptedAt + 7 days
  update state, timestamps and version = version+1
  append event certificate and audit entry in same transaction
  if next = DELETION_DUE: enqueue account-removal instruction atomically
  commit transaction; return certificate
```

All rejection paths roll back and release the lock. Proposed deadline semantics: store UTC instants and display Asia/Manila time. Interpret one week as 7 × 24 hours. Accept enrollment when its authoritative confirmation time is strictly before the deadline; at or after the deadline, the timeout wins if no timely enrollment exists. A shared transaction lock serializes the timeout and enrollment paths. Confirm this exact boundary with the academy; do not backdate events supplied by clients.

Worked lifecycle trace

REGISTERED receives a staff trial schedule, becomes TRIAL_SCHEDULED, attends assessment and becomes ASSESSMENT_PENDING. Approval at November 4, 2026, 10:00 Asia/Manila produces APPROVED_AWAITING_ENROLLMENT with a proposed cutoff of November 11 at 10:00. Confirmation on November 10 produces ENROLLED. In an alternative trace with no enrollment by the cutoff, DEADLINE_REACHED produces DELETION_DUE and triggers account removal. Replaying the same event ID returns its earlier result.

Conversion metric

$$Conversion_7 = 100 \times timely\ first\ enrollments\ /\ accepted\ trial\ participants$$

Group accepted trial participants by acceptance week and count each participant once. The numerator contains first enrollments confirmed within that participant's

seven-day window; the denominator contains all accepted participants in the same mature cohort. Rejected assessments are excluded. Return NOT_AVAILABLE when the denominator is zero or the cohort has not fully matured. Account deletion and approved data retention must be reconciled before retaining these statistics [1].

An FSM dispatch is O(1) with a fixed table and precomputed guard facts. Database locks, queries, audit storage, scheduled timeout discovery, notification delivery and account removal are separate costs. With E retained events, history occupies O(E). A full scan of N participants to find expired deadlines is O(N); indexed deadline discovery must be analyzed separately [4].

<!-- AI Assistance: ChatGPT assisted with event pseudocode, deadline semantics and the conversion definition. Team review is required before submission. No empirical results are claimed. -->

Task 2 Magic byte and image screening algorithm

The required signature algorithm inspects actual bytes rather than trusting a filename or browser MIME declaration. PNG begins with 89 50 4E 47 0D 0A 1A 0A [7]. JPEG begins with FF D8 followed by marker structure; FF D8 FF is a preliminary candidate check, not proof of a complete JPEG. Do not restrict JPEG to only JFIF or Exif application markers.

```
VALIDATE_IMAGE(stream, declaredMime, extension, rules):
  head <- read up to 8 bytes; reject truncated head
  if head[0:8] = [89,50,4E,47,0D,0A,1A,0A] in hexadecimal:
    detected <- image/png; allowedExtensions <- {.png}
  else if head[0:3] = [FF,D8,FF] in hexadecimal:
    detected <- image/jpeg; allowedExtensions <- {.jpg,.jpeg}
  else: return INVALID_SIGNATURE
  if normalized declaredMime != detected or
     lower-case extension not in allowedExtensions:
    return MIME_MISMATCH
  replay head into bounded quarantine stream; count actual bytes
  abort if actual byte count > MAX_BYTES; hash bytes incrementally
  at end reject if actual byte count < MIN_BYTES
  parse image with bounded marker/chunk lengths and pixel limit
  reject malformed/truncated/unsupported or animated image
  width,height <- validated image dimensions
  reject if width < MIN_WIDTH or height < MIN_HEIGHT
  decode with memory/time limits; reject decode failure
  compute grayscale variance and variance of Laplacian
  if contrast < MIN_CONTRAST or sharpness < MIN_SHARPNESS:
    return QUALITY_REVIEW with recapture guidance
  re-encode allowed format; drop unnecessary metadata/trailing data
  create random server filename; hand off to private storage
  hash stored output; record source and stored checksums separately
  record metadata; return VERIFIED_IMAGE
```

Proposed limits carried from the current spec: 5 KiB to 5 MiB inclusive and at least 400 × 400 pixels. Define KiB = 1,024 bytes and MiB = 1,048,576 bytes. Add a proposed 20-megapixel decode ceiling. Count bytes actually received; Content-Length alone is

untrusted. Use decoded grayscale variance for low contrast and Laplacian variance for blur; both are O(P) pixel scans. Thresholds must be calibrated on consented examples and recorded by document category. Until calibrated, route quality concerns for staff review rather than claiming reliable automated rejection [3, 8].

A matching signature cannot exclude polyglots, malicious payloads, or an unreadable ID. Structural decoding and re-encoding provide additional checks; private storage, authorization and staff document review remain necessary [8]. JPEG dimension markers may occur far beyond the first 32 bytes. The complete pipeline therefore is not O(1).

Quality calculation: after grayscale conversion, contrast is $\mathrm{Var}(I) = \mathrm{mean}(I^2) - \mathrm{mean}(I)^2$. For each interior pixel, define $L(x,y) = I(x-1,y) + I(x+1,y) + I(x,y-1) + I(x,y+1) - 4I(x,y)$; sharpness is $\mathrm{Var}(L)$. Accumulate sums and squared sums over pixels, then compare with the calibrated category thresholds. These are proposed screening heuristics, not proof of document authenticity or text legibility.

<!-- AI Assistance: ChatGPT assisted with signature pseudocode and the proposed quality-screening policy. Team review is required before submission. No empirical results are claimed. -->

Task 3 Theoretical complexity and scalability

CART derivation for this implementation

Let n be training rows, d numeric features, H the maximum tree depth, h the realized depth and T the number of nodes. At a node with m rows, counting and partitioning cost O(m). For each feature, sorting temporary pairs costs O(m log m) and the prefix scan costs O(m), giving O(d m log m). These bounds analyze the specified implementation, which sorts again at each node [4, 5].

$$C(m) = C(mL) + C(mR) + O(d\, m \log m)$$

At a fixed depth, node subsets are disjoint and sum to at most n, so their combined work is O(d n log n). Summing over h levels gives O(d n h log n), bounded by O(d n H log n). For balanced splitting without a fixed cap, $h = O(\log n)$, hence $O(d\, n \log^2 n)$. A highly skewed unrestricted tree yields the conservative bound $O(d\, n^2 \log n)$. Do not claim O(d n log n) for repeated per-node sorting.

A pure root or $n < 2L$ stops after O(n) core counting; input validation adds O(nd). For mixed data, no average-case bound is asserted without a distribution assumption. With fixed $H = 8$ and $d = 5$, the upper bound simplifies to O(n log n). Prediction traverses h nodes: best O(1), worst O(h), at most O(H). A balanced unrestricted tree has O(log n) path length; an unrestricted skewed tree can require O(n).

Auxiliary training memory is O(n + h + T), excluding the read-only O(nd) input. It comprises one global row-index array, one reusable O(n) feature-pair scratch buffer, a depth-h recursion stack, and T output nodes. Discard scratch before recursion and partition the global index range in place. $T \le \min(2\lfloor n/L \rfloor - 1, 2^{H+1} - 1)$. Prediction uses O(1) traversal space or O(h) when returning the explanation path.

Supporting engines and scale dimensions

Operation or dimension	Cost or parameter	Implication
FSM table dispatch	O(1) time and transient space	Excludes persistence, history and deletion.
Signature only	O(k), k ≤ 8; O(1) space	Constant-size rejection gate.
Full image screening	O(B + P) time; O(C + P) space	B bytes, P pixels, C fixed stream buffer; bounded decoding still scales with pixels.
Volume	n = 100, 1,000, 5,000 synthetic rows	Offline training; bound depth and payload size.
Velocity	Proposed stress sweep 1, 10, 25 requests/s	Measure inference latency separately from training and storage.
Variety	Snapshots, events and JPEG/PNG bytes	Version schemas; normalize dates, guards and images.

<!-- AI Assistance: ChatGPT assisted with the implementation-specific complexity derivations and scalability assumptions. Team review is required before submission. No empirical results are claimed. -->

Task 4 Decoupled architecture

Keep the algorithmic core independent of ORM models, HTTP request objects and UI components. Use serializable data transfer objects and versioned model artifacts. The initial design remains within the existing TypeScript and Express backend; it introduces no new framework or external service. Offline training belongs outside

the request path. If measured CPU load requires parallelism, Node.js worker threads provide a separate execution context for JavaScript CPU work [11].

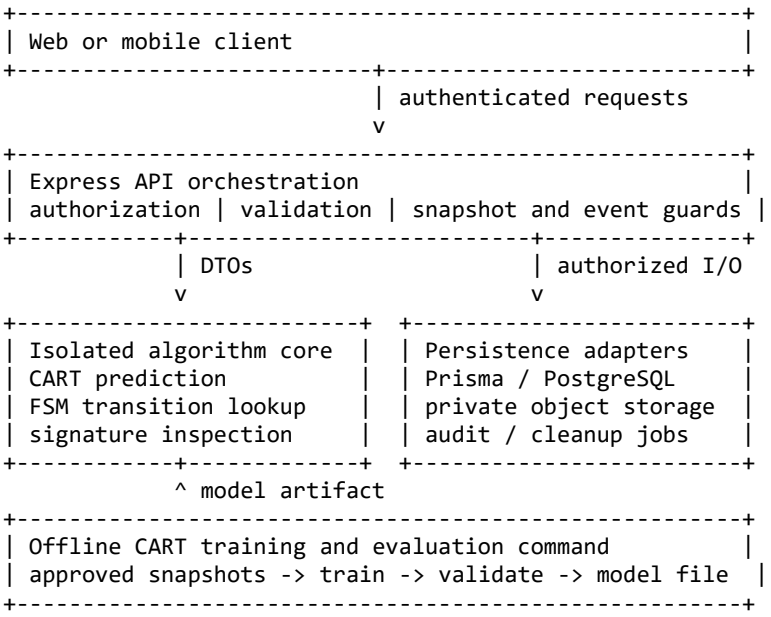


Figure 1. Component boundaries. The core receives validated DTOs; persistence adapters perform database and storage operations. Upload decoding uses a bounded adapter around the core signature check. Training is an owner-invoked offline operation, with a versioned artifact promoted only after held-out evaluation.

Module responsibilities

Proposed modules: apps/api/src/modules/analytics/algorithm/ for CART; apps/api/src/modules/participants/lifecycle/ for FSM; apps/api/src/modules/documents/validation/ for signatures. These are planning paths, not claims that files already exist. Controllers authorize requests and load authoritative facts. Pure functions compute certificates. Orchestration atomically persists state and audit entries and handles cleanup or storage failure.

Staff alone receive participant-level risk explanations. Mobile clients receive their own authorized lifecycle and document feedback. No image, name, medical detail or guardian data is sent to CART. The client cannot provide a trusted assessment result, fee-confirmation flag or lifecycle state. CPU-heavy image decoding must have explicit memory and time limits [8].

<!-- AI Assistance: ChatGPT assisted with the component architecture and proposed module boundaries. Team review is required before submission. No empirical results are claimed. -->

Task 4 API request and response contracts

Proposed staff-only POST /api/v1/algorithm/solve uses operation CHURN_PREDICT. The API looks up the participant's server-side snapshot and the active model; the

client does not submit predictor values or choose a model file. The internal DTO shown below is created after authorization. Document these contracts with OpenAPI 3.1 [9]. HTTP semantics follow RFC 9110 [10].

```
External request
{"requestId":"req-demo-001", "operation":"CHURN_PREDICT",
 "participantId":"participant-demo-001",
 "asOf":"2026-11-04T02:00:00Z"}

Internal engine DTO
{"modelVersion":"cart-demo-v1", "features":{
 "daysSinceAttendance":12, "unexcusedAbsenceStreak":3,
 "sessionsRemaining":2, "attendanceRateLast10":0.4,
 "daysSinceEnrollment":45}}

Illustrative response for an illustrative approved model
{"requestId":"req-demo-001", "status":"SCORED",
 "mode":"CART", "modelVersion":"cart-demo-v1",
 "riskLabel":"HIGH_RISK", "leafFrequency":0.8,
 "leafSupport":10, "path":[
   {"feature":"unexcusedAbsenceStreak", "threshold":2,
    "branch":"RIGHT"}],
 "executionMetrics":{"nodesVisited":2,"runtimeMs":0.0}}
```

The numerical response is an example, not a measured result. runtimeMs is a nonnegative measured value in an actual implementation. An absent model returns status NOT_SCORED, mode RULE_BASELINE, probability null, and a named rule flag when applicable. Missing snapshot features return NOT_SCORED with a safe reason code.

Field or result	Required validation and behavior
External body	Exactly four required fields; requestId and participantId are strings of 1–64 ASCII letters, digits or hyphens; operation is CHURN_PREDICT; asOf is an RFC 3339 UTC date-time not in the future. Reject additional fields in this proposed endpoint contract.
Internal features	Exactly five finite fields with Task 1 ranges; enforce participant cohort and model feature-version compatibility.
Response	Required requestId, status, mode and executionMetrics. SCORED additionally requires modelVersion, riskLabel, leafFrequency in [0,1], positive leafSupport and bounded path.
HTTP outcomes	200 for SCORED or NOT_SCORED; 400 malformed request; 401 unauthenticated; 403 unauthorized; 404 unavailable authorized record; 409 stale FSM event; 413 oversized upload; 415 unsupported media; 500 generic failure.

Lifecycle events use an authenticated staff endpoint with eventId, participantId, expectedVersion, type and event-specific evidence IDs; timestamps and guard facts are server-derived. Uploads use multipart/form-data with category VALID_ID and file bytes, not JSON/base64. Respond with a stable status and recovery guidance; never expose raw decoder exceptions [8–10].

<!-- AI Assistance: ChatGPT assisted with the API examples, field constraints and response semantics. Team review is required before submission. No empirical results are claimed. -->

Task 5 Agile backlog and acceptance

User stories

As academy staff, I want an explainable churn-risk result from authorized attendance and enrollment snapshots so that I can prioritize follow-up without changing participant eligibility.

As an academy administrator, I want assessment and enrollment events validated by a lifecycle FSM and uploaded IDs checked against their true image format so that records remain consistent with the revised flow and unsafe or unreadable uploads can be corrected.

Ticket and proposed module	Implementation steps	Acceptance and security
DEV-ALGO-101 analytics/algorithm/	1 Define five-feature schema and mature label. 2 Create participant/time splits. 3 Implement Gini prefix scan and in-place index partition. 4 Implement bounded training and prediction path. 5 Version and evaluate artifacts.	Set initial bestGain = 0 and require gain > bestGain + ε. Pure, constant-feature and repeated-value cases terminate. Depth ≤ 8, each leaf ≥ 5. Match hand-computed Gini splits. Staff-only inference.
DEV-ALGO-102 participants/lifecycle/	1 Reconcile approved state table. 2 Implement pure lookup and guards. 3 Add event idempotency and version checks. 4 Persist transaction and audit. 5 Implement deadline discovery and account removal after scope approval.	Reject enrollment before trial approval. Cover rejection, timely enrollment and exact cutoff. Race enrollment against timeout. Replays cause no extra mutation. Account removal completion is verified separately.
DEV-ALGO-103 documents/validation/	1 Implement prefix signatures. 2 Cross-check MIME and extension. 3 Enforce actual byte limits. 4 Add bounded decode and quality scans. 5 Re-encode and store with random key after authorization.	Reject disguised script, mismatch, truncated PNG/JPEG, oversized stream and extreme pixel dimensions. Accept valid JPEG beyond JFIF/Exif. No public ID URLs. Storage failures permit a safe retry without duplicate records.

Dependencies and completion criteria

Owner approval of specification reconciliation precedes implementation of changed lifecycle behavior or the primary replacement. Dataset readiness and validated labels precede predictive deployment. Calibrated document-quality thresholds precede automatic quality rejection. Each ticket must document actual file paths, test commands, benchmark artifacts and any unresolved policy. The report does not claim these prototype modules have been implemented or committed.

<!-- AI Assistance: ChatGPT assisted with the backlog, user stories and acceptance criteria. Team review is required before submission. No empirical results are claimed. -->

Task 5 Empirical validation plan

Correctness and model evaluation

Use deterministic synthetic fixtures for algorithm mechanics and timing. Publish the generator seed, schema and generation rules with the prototype; no external dataset is claimed. Include empty data, pure labels, equal feature values, tied gains, minimum leaves, maximal depth, missing values and very uneven partitions. Compare small trees against manually calculated splits. Synthetic results must remain labeled synthetic.

For real predictive evaluation, divide participants into disjoint training, validation and test groups, and order their snapshot windows chronologically. Require the latest training outcome window to end before validation snapshots begin and similarly before test snapshots. Discard rows that violate this gap; never use a participant in more than one partition. Tune settings on validation only and report the test set once. Do not construct labels from future data that would have been unavailable at training time [5].

Report confusion matrix, positive-class precision, recall, F1 and PR-AUC against the majority and rule baselines; report sample counts, class prevalence and uncertainty. Accuracy alone is insufficient when lapse cases are rare. If the model does not improve a pre-agreed follow-up metric on held-out academy data, keep rule mode and collect more data. No measured quality or accuracy threshold is claimed in this milestone [5, 6].

Proposed automated benchmark assertions

Target	Fixture and assertion	Measurement boundary
Training	n = 5,000, d = 5, H = 8, L = 5: median < 2 s; record peak memory.	Includes sorting, counts, partitioning and artifact construction; excludes dataset generation and file I/O.
Inference	10,000 valid predictions: median batch < 250 ms; each path length ≤ 8.	Model preloaded; includes explanation-path construction; excludes network and DB.
FSM	10,000 pure table dispatches: median batch < 250 ms; all invalid pairs rejected.	Exclude persistence; separately measure real transaction and timeout behavior.
Signatures	10,000 eight-byte checks: median batch < 250 ms.	Exclude full streaming, decoding and storage; report them separately by B and P.

These are initial engineering targets, not observations. On an owner-controlled fixed runner, run one warm-up and at least ten measured repetitions per size (100, 1,000, 5,000 training rows). Record median and p95, Node version, OS, CPU, RAM, feature count, depth, seed, whether garbage collection is controlled, and exact timing

boundaries. Measure concurrent HTTP latency separately. Any target revision must retain the original result and explain the change.

Owner-run review also covers the FSM cutoff in Asia/Manila time, no-model behavior, invalid image recapture guidance, and transactional retries. Under the repository execution boundary, the owner runs all application test suites; none were run to prepare this proposal.

<!-- AI Assistance: ChatGPT assisted with the correctness cases, leakage controls and benchmark plan. Team review is required before submission. No empirical results are claimed. -->

Task 6 Academic review and defense

1 Separation of heavy computation

Orchestration handles ORM access, authorization and persistence. The core receives plain arrays or DTOs for reproducible computation. Training runs offline; image decoding needs resource limits. Measure request blocking before proposing worker execution or an external service [8, 11].

2 Exponential complexity and production viability

The specified greedy construction is polynomial: O(d n H log n) with bounded depth. Depth, minimum-leaf and gain constraints limit work and overfitting. It avoids exhaustive tree search and makes no global-optimum claim. Global tree optimization would require a different formulation and analysis [4, 5].

3 Exact algorithms and heuristics

An exact optimizer guarantees the optimum for a defined model. CART chooses local splits and can miss a better complete tree. Inference deterministically follows that tree, but the future outcome remains uncertain. FSM and signature checks apply fixed rules; they establish neither predictive accuracy nor universal file safety [5, 7, 8].

4 Empirical profiling and asymptotic analysis

Big-O describes resource growth under stated input sizes and cost assumptions. Benchmarks measure a particular runtime, machine and dataset. Neither proves the other: a fast trial cannot establish a complexity bound, and a polynomial bound cannot establish a latency target. Report both with explicit measurement boundaries [4].

5 Cold start and sparse data

CART needs mature labeled snapshots and held-out evaluation. Until ready, use NOT_SCORED and the labeled rule baseline with no probability. Reject missing predictors. Synthetic data demonstrates mechanics only. Participant-disjoint

temporal partitions and minimum leaves help control leakage and sparsity; representative academy data remains necessary [5, 6].

6 Edge cases and host-service resilience

Validate features and model versions; stop at pure or unsplittable nodes. Reject invalid FSM pairs and stale versions; make retries idempotent. Bound upload bytes and pixels, reject truncation and return safe errors. Keep predictions separate from access decisions. The owner verifies correctness, transaction races and resource limits [8–10].

<!-- AI Assistance: ChatGPT assisted with the academic defense explanations. Team review is required before submission. No empirical results are claimed. -->

References

IEEE numerical citations are used throughout. Public theoretical and technical sources below date from 2021–2026. The supplied course handout has no publication date; it is identified as an internal assignment source rather than assigned a fabricated date. Web sources were checked November 4, 2026.

[1] Oak Sports Academy, “Oak Sport Academy System Flow Revised,” Nov. 4, 2026, 8 pp. Supplied project PDF, attributed to @nath.

[2] ALGOCOM / SOFTENG, “PBL Milestone 1 Algorithm Architecture Specification and Software Engineering Proposal,” course assignment handout, undated, supplied DOCX, consulted Nov. 4, 2026.

[3] Oak Sports Academy project team, “Final Project Technical Specification and AI Development Instructions,” specifications.md, working repository copy consulted Nov. 4, 2026, especially Sec. 15. https://github.com/hex-xel/oak-sports-academy

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

[5] S. Raschka, Y. Liu, and V. Mirjalili, Machine Learning with PyTorch and Scikit-Learn. Birmingham, UK: Packt, 2022, chs. 3 and 6. ISBN 9781801819312. Author companion repository: https://github.com/rasbt/machine-learning-book

[6] A. Gonzalez-Nucamendi, J. Noguez, L. Neri, V. Robledo-Rella, and R. M. G. García-Castelán, “Predictive analytics study to determine undergraduate students at risk of dropout,” Frontiers in Education, vol. 8, Art. 1244686, 2023, doi: 10.3389/feduc.2023.1244686. https://doi.org/10.3389/feduc.2023.1244686

[7] W3C, Portable Network Graphics PNG Specification Third Edition, W3C Recommendation, Jun. 24, 2025. https://www.w3.org/TR/2025/REC-png-3-20250624/

[8] OWASP Foundation, Application Security Verification Standard, ver. 5.0.0, May 30, 2025, especially file-handling requirements. https://asvs.owasp.org/

[9] OpenAPI Initiative, OpenAPI Specification, ver. 3.1.0, Feb. 15, 2021. https://spec.openapis.org/oas/v3.1.0.html

[10] R. Fielding, M. Nottingham, and J. Reschke, Eds., HTTP Semantics, RFC 9110, IETF, Jun. 2022. https://www.rfc-editor.org/rfc/rfc9110.html

[11] Node.js contributors, “Worker threads,” Node.js v26.13.0 documentation, consulted Nov. 4, 2026. https://nodejs.org/api/worker_threads.html

[12] P. Linz and S. H. Rodger, An Introduction to Formal Languages and Automata, 7th ed. Jones & Bartlett Learning, 2023, ch. 2. ISBN 9781284231601. https://www.jblearning.com/catalog/productdetails/9781284231601

Appendix Specification reconciliation and review

Changes proposed for owner approval

Current baseline	Revised proposal and reason
Section 15 Algorithm 1 is CC-WIS schedule recommendation.	Replace it with CART for the professor-required churn capability; the revised flow has no preference-scored recommendation feature.
FSM permits REGISTERED directly to ENROLLED and post-trial inactivity after 14 days.	Require attended assessment and staff acceptance before enrollment. Enforce account deletion for accepted non-enrollees after one week [1].
FSM includes maximum two trial days and several fixed inactivity timers.	Treat trial entitlement and general inactivity timers as unconfirmed; do not silently carry them into the revised flow.
Image pipeline is described as constant time and dimensions come from a short header.	Limit the O(1) claim to signature bytes; analyze JPEG parsing, decoding, quality scans, hashing and streaming separately.
Current login permits verified ACTIVE accounts; revised flow says completed trial and enrolled.	Resolve whether accepted but unenrolled accounts may authenticate or must use a separate enrollment path. This report makes no access-control change.

Client decisions still required

The source flow itself leaves white-belt uniform pricing, black-belt options, make-ups for students without modules, spelling and computed age open [1, p. 8]. This report does not resolve those business questions. Additional approval is needed for the precise seven-day cutoff, account deletion scope and reporting retention, the 30-day churn proxy, training readiness, baseline absence threshold and quality-screening calibration.

Submission review

Included: Tasks 1–6, team metadata, concrete algorithms and pseudocode, worked examples, complexity derivations, architecture, API contract, two user stories, implementation tickets, benchmark targets, references and AI disclosures.

Before submission, the team must review the AI-assisted content, confirm proposed policies, approve specification reconciliation, implement and measure benchmarks, verify repository access, confirm the submission date and prepare the separate deck. Application test suites were not run. No application code or source specification changed.